

Competitive Programming

Section 71: Discrete Convexity And Lagrangian Optimization

Competitive Programming Course

Table of contents

1	Convexity as ordered marginal decisions	3
2	Slope trick through isotonic L1 repair	4
3	Lagrangian relaxation: price the hard count	5
4	Huge separable convex allocation	6
5	Worked trace: the heap is a derivative, not a bag of values	7
6	Worked trace: exact-count penalties need intervals	8
7	Worked trace: threshold ties	9
8	C++ and Python implementation notes	10
9	Proof habits	11
10	Recognition checklist	12
11	Exercises	13

1 Convexity as ordered marginal decisions

Convex optimization in contests often means this discrete statement:

Taking the next unit becomes no cheaper as more units are already taken.

Once the marginal costs are sorted inside every choice, a large state space can collapse into heaps, a threshold search, or a penalty parameter.

2 Slope trick through isotonic L1 repair

After processing a prefix, regard its optimal cost as a convex piecewise-linear function of the last chosen value. Adding $|x - a[i]|$ inserts a slope change at $a[i]$. Requiring the sequence to be nondecreasing discards the part of the function that would need a decreasing final value.

The compact implementation stores left breakpoints in a max-heap. For each x , insert it twice, remove the largest breakpoint y , and add $y - x$ to the answer. One copy represents the new absolute-value kink; the second allows the monotonicity projection to move a previous kink down to x . Because x was just inserted, $y \geq x$.

The heap is not an unexplained trick: it is a compressed derivative. Its elements are exactly the points where the slope of the current convex value function increases.

3 Lagrangian relaxation: price the hard count

Suppose a feasible object has original cost $\text{cost}(T)$ and uses $r(T)$ red edges, but we require $r(T)=k$. Add a price λ per red edge and solve

```
minimize cost(T) + lambda * r(T).
```

As λ increases, optimal solutions use no more red edges. This monotonic response permits binary search. For a fixed price, an MST is still obtained by Kruskal after adding λ to every red edge.

At equal modified weights, run Kruskal once preferring red and once preferring blue. These give the maximum and minimum red counts among modified-optimal trees. The graphic-matroid exchange property ensures all intermediate counts are attainable without changing modified cost. If k lies in that interval,

```
exact answer = modified optimum - lambda*k.
```

Always test global feasibility first: the minimum and maximum possible red counts among all spanning trees must bracket k .

4 Huge separable convex allocation

Machine i has marginal cost

$$\text{delta}_i(j) = a_i \cdot (2j-1) + b_i$$

for its j -th unit. Each machine contributes a sorted arithmetic progression; the answer is the sum of the globally smallest K marginals. Expanding them is impossible when capacities are enormous.

Binary-search the smallest threshold T for which at least K marginals are at most T . Count a machine's qualifying marginals with one division, capped by its capacity. Sum every marginal strictly below T , then take exactly the remaining number equal to T . Strict versus non-strict thresholding is what handles ties correctly.

5 Worked trace: the heap is a derivative, not a bag of values

For unweighted targets 8,3, insert the first breakpoint twice and remove one largest copy. One copy of 8 remains. At target 3, insert 3 twice; the largest breakpoint is 8, so moving it down to 3 costs $8-3=5$. The remaining breakpoints summarize the derivative of the optimum after enforcing that the two repaired values are nondecreasing.

For a weight w , the absolute-value term changes the derivative by $2w$ at its target. Conceptually insert $2w$ copies and remove w largest copies. Compressing equal copies into `(position,mass)` records makes the running time depend on the number of targets, not the sum of weights.

6 Worked trace: exact-count penalties need intervals

At one penalty λ , several spanning trees can have equal modified cost. One may use two red edges and another four. If the required count is three, returning only whichever tree Kruskal happens to construct can falsely report failure. Run opposite tie rules to obtain the minimum and maximum red counts among modified-optimal trees. The feasible count at this penalty is an interval, and three lies inside it.

This is the general lesson for Lagrangian relaxation with an exact discrete count: binary-searching a price is insufficient until ties and attainability between extreme optimal counts are proved.

7 Worked trace: threshold ties

Suppose the marginal sequences contribute values

```
machine A: 1, 5, 9  
machine B: 3, 5, 7
```

For $K=4$, the smallest threshold with at least four marginals at most it is 5. There are two values strictly below 5 and two equal to 5, so take both ties. For $K=3$, take the two strict values and only one of the ties. Summing all values $\leq T$ would take too many units.

8 C++ and Python implementation notes

In weighted breakpoint heaps, store multiplicities rather than repeating a value w times. Total weight fits in 64 bits under the stated bounds, but `weight * distance` and aggregate costs may need `__int128`; Python integers grow automatically. Write and test a small decimal printer for 128-bit output.

For penalty search, keep modified objective and chosen-count information in one return value so tie behavior is explicit. For threshold search, implement both `count(<T)` and `count(<=T)` deliberately; deriving one by carelessly subtracting one fails at the numeric boundary.

9 Proof habits

- State the convex value function represented by a slope-trick structure and what each stored breakpoint means.
- For a Lagrangian price, prove monotonicity of the resource count and explain why the desired exact count is attainable at the chosen boundary.
- For marginal selection, prove prefix closure inside every source sequence and then prove that the threshold construction chooses exactly K values.

10 Recognition checklist

1. Write the slow DP or allocation problem.
2. Ask whether its value function is convex or its marginal choices are sorted.
3. Identify the compressed representation: breakpoints, a penalty, or a threshold count.
4. Prove monotonicity and tie behavior before binary searching.
5. Keep the original objective separate from the penalized one.

11 Exercises

A develops the breakpoint view of slope trick. B requires an exact-count penalty with rigorous tie handling. C replaces an enormous allocation state space by a threshold over sorted marginal sequences. D is blind: weights and hard value bounds must be incorporated without expanding either the weights or the value domain.